

From Fixed Expressions to Formulas

Variables

So far, we have looked at type-checking, evaluating (in a value domain) and calculating (in abstract syntax) expressions. However, “programming” makes sense when one performs the same calculation on *different* inputs. In that sense, our toy language compiler or a definitional interpreter are programs that can work on different expressions as input — however, the toy language is not yet a “programming language”. The next step we will take is to make some parts of expressions vary. Mathematicians do this all the time: for example, they calculate a *formula* $b^2 - 4ac$ for different values of a, b, c . The expressions a, b, c are called input *variables* since they can (each) be given *various* values.

So our first step is to extend the toy language with a new case of expressions — namely, variables. Assume that we have a (denumerable) set of identifiers $\mathcal{X}$ (assume also that these identifiers are different from other symbols in the language), with $x, y, z, x_1, y_1, \dots \in \mathcal{X}$ *representing* typical variables. (We call $x, y, z, x_1, y_1, \dots$ “meta-variables” — the actual variables in the language will be particular strings).

Following an “abstract grammatical notation” to characterise the inductively defined set of expressions (i.e., abstract syntax), we can write

$$\underline{e} \in \text{Exp} ::= \underline{n} \mid \underline{\mathbb{T}} \mid \underline{\mathbb{F}} \mid \underline{x} \mid \underline{e_1 + e_2} \mid \underline{e_1 * e_2} \mid \underline{e_1 \wedge e_2} \mid \underline{e_1 \vee e_2} \mid \underline{\neg e_1} \mid \underline{e_1 = e_2} \mid \underline{e_1 > e_2}$$

We extend the OCaml encoding of the language

```
type exp = Num of int | Bl of myBool
         | V of string (* variables *)
         | Plus of exp * exp | Times of exp * exp
         | And of exp * exp | Or of exp * exp | Not of exp
         | Eq of exp * exp | Gt of exp * exp

;;
```

by redefining the type `exp` to include another case (namely, variables) represented using a constructor `V` that takes a string (the name of a particular variable in the toy language) as an argument.

The functions `ht` and `size` are amended as follows, mapping all variables to have height 0, and size 1.

```
let rec ht e = match e with
  Num n   -> 0
| Bl b    -> 0
| V x     -> 0
| Plus (e1, e2) -> 1 + (max (ht e1) (ht e2))
| Times (e1, e2) -> 1 + (max (ht e1) (ht e2))
| And (e1, e2)  -> 1 + (max (ht e1) (ht e2))
| Or (e1, e2)   -> 1 + (max (ht e1) (ht e2))
| Not e1        -> 1 + (ht e1)
| Eq (e1, e2)   -> 1 + (max (ht e1) (ht e2))
| Gt (e1, e2)   -> 1 + (max (ht e1) (ht e2))

;;
```

and

```

let rec size e = match e with
  Num n   -> 1
| Bl b    -> 1
| V x    -> 1
| Plus (e1, e2) -> 1 + (size e1) + (size e2)
| Times (e1, e2) -> 1 + (size e1) + (size e2)
| And (e1, e2)  -> 1 + (size e1) + (size e2)
| Or (e1, e2)   -> 1 + (size e1) + (size e2)
| Not e1        -> 1 + (size e1)
| Eq (e1, e2)   -> 1 + (size e1) + (size e2)
| Gt (e1, e2)   -> 1 + (size e1) + (size e2)
;;

```

Types and typing rules

Assume we have a set *Typ* of types. Let $\tau, \tau_1, \tau' \in \text{Typ}$ represent typical types (again, τ, τ_1, τ' are “meta-variables” ranging over types). So far, we have considered Int and Bool as members of *Typ*. (This will be extended as we proceed later). We have seen earlier typing rules that associate numeric expressions with type Int and boolean expressions with type Bool. But what type should we give variables?

Type Assumptions. A *typing assumption* $\Gamma \in \mathcal{X} \rightarrow_{fin} \text{Typ}$ is a finite-domain function from variables to *types*, that is, it associates a type with any variable in its domain.

If Γ is a type assumption, $\underline{x} \in \mathcal{X}$ a variable, and $\tau \in \text{Typ}$ a type, we write $\Gamma[\underline{x} : \tau]$ to denote the type assumption that associates the type τ to variable $\underline{x}$ and for other variables in $\text{dom}(\Gamma)$, associates them to types exactly as Γ would.

This notion generalises as follows:

If Γ, Γ_1 are type assumptions, then $\Gamma[\Gamma_1]$ denotes the type assumption defined as

$\Gamma[\Gamma_1](\underline{x}) = \Gamma_1(\underline{x})$ if $\underline{x} \in \text{dom}(\Gamma_1)$;
 $\Gamma[\Gamma_1](\underline{x}) = \Gamma(\underline{x})$ if $\underline{x} \in \text{dom}(\Gamma) - \text{dom}(\Gamma_1)$;
 and *undefined* if $\underline{x} \notin \text{dom}(\Gamma) \cup \text{dom}(\Gamma_1)$.

(This is in fact, the standard notion of one finite domain function being augmented by another.)

The “has type” relation is now modified to carry a type assumption, written to the left of the “turnstile”, to handle the presence of variables within expressions, and by adding a rule to deal with the base case of variables. Note that each statement is modified to read as

$\Gamma \vdash _ : _$

(NumT) $\frac{}{\Gamma \vdash \underline{n} : \text{Int}}$ All numerals $\underline{n}$
 have type Int for *any* Γ

(BoolT) $\frac{}{\Gamma \vdash \underline{b} : \text{Bool}}$ All boolean constants $\underline{b}$ have type Bool for *any* Γ

(VarT) $\frac{}{\Gamma \vdash \underline{x} : \Gamma(\underline{x})}$ A variable has the type it is *assumed* to have.

$$\text{(PlusT)} \frac{\Gamma \vdash \underline{e_1} : \underline{\text{Int}} \quad \Gamma \vdash \underline{e_2} : \underline{\text{Int}}}{\Gamma \vdash \underline{e_1 + e_2} : \underline{\text{Int}}}$$

All addition expressions $\underline{e_1 + e_2}$ have type $\underline{\text{Int}}$, provided the subexpressions $\underline{e_1}$ and $\underline{e_2}$ both have type $\underline{\text{Int}}$ under the *same* type assumptions Γ .

$$\text{(TimesT)} \frac{\Gamma \vdash \underline{e_1} : \underline{\text{Int}} \quad \Gamma \vdash \underline{e_2} : \underline{\text{Int}}}{\Gamma \vdash \underline{e_1 * e_2} : \underline{\text{Int}}}$$

All multiplication expressions $\underline{e_1 * e_2}$ have type $\underline{\text{Int}}$, provided the subexpressions $\underline{e_1}$ and $\underline{e_2}$ both have type $\underline{\text{Int}}$ under the *same* type assumptions Γ .

$$\text{(NotT)} \frac{\Gamma \vdash \underline{e_1} : \underline{\text{Bool}}}{\Gamma \vdash \underline{\neg e_1} : \underline{\text{Bool}}}$$

All negation expressions $\underline{\neg e_1}$ have type $\underline{\text{Bool}}$, provided the subexpressions $\underline{e_1}$ have type $\underline{\text{Bool}}$ under the *same* type assumptions Γ .

$$\text{(AndT)} \frac{\Gamma \vdash \underline{e_1} : \underline{\text{Bool}} \quad \Gamma \vdash \underline{e_2} : \underline{\text{Bool}}}{\Gamma \vdash \underline{e_1 \wedge e_2} : \underline{\text{Bool}}}$$

All conjunction expressions $\underline{e_1 \wedge e_2}$ have type $\underline{\text{Bool}}$, provided the subexpressions $\underline{e_1}$ and $\underline{e_2}$ both have type $\underline{\text{Bool}}$ under the *same* type assumptions Γ .

$$\text{(OrT)} \frac{\Gamma \vdash \underline{e_1} : \underline{\text{Bool}} \quad \Gamma \vdash \underline{e_2} : \underline{\text{Bool}}}{\Gamma \vdash \underline{e_1 \vee e_2} : \underline{\text{Bool}}}$$

All disjunction expressions $\underline{e_1 \vee e_2}$ have type $\underline{\text{Bool}}$, provided the subexpressions $\underline{e_1}$ and $\underline{e_2}$ both have type $\underline{\text{Bool}}$ under the *same* type assumptions Γ .

$$\text{(EqT)} \frac{\Gamma \vdash \underline{e_1} : \underline{\tau} \quad \Gamma \vdash \underline{e_2} : \underline{\tau}}{\Gamma \vdash \underline{e_1 = e_2} : \underline{\text{Bool}}}$$

All equality expressions $\underline{e_1 = e_2}$ have type $\underline{\text{Bool}}$, provided the subexpressions $\underline{e_1}$ and $\underline{e_2}$ both have type $\underline{\tau}$ under the *same* type assumptions Γ .

$$\text{(GtT)} \frac{\Gamma \vdash \underline{e_1} : \underline{\text{Int}} \quad \Gamma \vdash \underline{e_2} : \underline{\text{Int}}}{\Gamma \vdash \underline{e_1 > e_2} : \underline{\text{Bool}}}$$

All greater-than expressions $\underline{e_1 > e_2}$ have type $\underline{\text{Bool}}$, provided the subexpressions $\underline{e_1}$ and $\underline{e_2}$ both have type $\underline{\text{Int}}$ under the *same* type assumptions Γ .

Modifying the definitional interpreter

How does the definitional interpreter change? Well, what is the value of a variable? Whatever value we give to the variable by a “valuation”, i.e., a function $\rho \in \mathcal{X} \rightarrow \mathbb{V}$ from variables to values in the set of values $\mathbb{V}$. So each equation for *eval* takes ρ as an additional argument:

$eval[\![\underline{n}]\!] \rho = n$
 $eval[\![T]\!] \rho = true$ and $eval[\![F]\!] \rho = false$

$eval[\![\underline{x}]\!] \rho = \rho(\underline{x})$

$eval[\![\underline{e_1} + \underline{e_2}]\!] \rho = (eval[\![\underline{e_1}]\!] \rho) + (eval[\![\underline{e_2}]\!] \rho)$

$eval[\![\underline{e_1} * \underline{e_2}]\!] \rho = (eval[\![\underline{e_1}]\!] \rho) \times (eval[\![\underline{e_2}]\!] \rho)$

(where $+$, $\times$ represent integer addition and multiplication).

$eval[\![\underline{e_1} \wedge \underline{e_2}]\!] \rho = (eval[\![\underline{e_1}]\!] \rho) \ \&\& \ (eval[\![\underline{e_2}]\!] \rho)$

$eval[\![\underline{e_1} \vee \underline{e_2}]\!] \rho = (eval[\![\underline{e_1}]\!] \rho) \ || \ (eval[\![\underline{e_2}]\!] \rho)$

$eval[\![\neg \underline{e_1}]\!] \rho = not \ (eval[\![\underline{e_1}]\!] \rho)$

(where $\&\&$, $||$, not represent boolean conjunction, disjunction and negation).

$eval[\![\underline{e_1} = \underline{e_2}]\!] \rho = (eval[\![\underline{e_1}]\!] \rho) =^? (eval[\![\underline{e_2}]\!] \rho)$

$eval[\![\underline{e_1} > \underline{e_2}]\!] \rho = (eval[\![\underline{e_1}]\!] \rho) >^? (eval[\![\underline{e_2}]\!] \rho)$

(where $=^?$, $>^?$ represent equality and greater-than comparisons on integers).

The Modified Definitional Interpreter in OCaml

```

type values = N of int | B of bool;;

let rec eval e rho = match e with
  | Num n  -> N n
  | B1 b   -> B (myBool2bool b)
  | V x    -> rho x
  | Plus (e1, e2)  -> let N n1 = (eval e1 rho)
                        and N n2 = (eval e2 rho)
                        in N (n1 + n2)
  | Times (e1, e2) -> let N n1 = (eval e1 rho)
                        and N n2 = (eval e2 rho)
                        in N (n1 * n2)
  | And (e1, e2)   -> let B b1 = (eval e1 rho)
                        and B b2 = (eval e2 rho)
                        in B (b1 && b2)
  | Or (e1, e2)    -> let B b1 = (eval e1 rho)
                        and B b2 = (eval e2 rho)
                        in B (b1 || b2)
  | Not e1         -> let B b1 = (eval e1 rho) in B (not b1)
  | Eq (e1, e2)    -> let a1 = (eval e1 rho)
                        and a2 = (eval e2 rho)
                        in B (a1 = a2)
  | Gt (e1, e2)    -> let N n1 = (eval e1 rho)
                        and N n2 = (eval e2 rho)
                        in B (n1 > n2)
;;

```

Note that the OCaml interpreter is able to effortlessly *infer* the type of the modified eval function:

```
val eval : exp -> (string -> values) -> values = <fun>
```

Big Step (Natural, Kahn-style) Operational Semantics

The big-step (Kahn-style) operational semantics also needs to be modified. The calculates relation now needs to return an answer when the input expression is a variable. What answer? Whatever answer the variable is bound to. So we need a *data structure that associates variables to canonical answers*. Let us call this a “table”, which is nothing but a *finite-domain function* $\gamma \in \mathcal{X} \rightarrow_{fin} Ans$. Why “finite-domain”? Because a calculator has to operate with finite data structures and not mathematical abstractions such as valuations (which can be infinite).

Accordingly, we modify the calculates relation by introducing a table in each of the rules. And we add a rule to deal with the case of variables. To highlight that the table does not change during the calculation process, we place the table to the left of a turnstile.

$$(\text{CalcNum}) \frac{}{\gamma \vdash \underline{n} \Rightarrow \underline{n}} \text{ for any } \gamma$$

$$(\text{CalcBool}) \frac{}{\gamma \vdash \underline{b} \Rightarrow \underline{b}} \text{ for any } \gamma$$

$$(\text{CalcVar}) \frac{}{\gamma \vdash \underline{x} \Rightarrow \gamma(\underline{x})} \text{ provided } \underline{x} \in \text{dom}(\gamma)$$

$$(\text{CalcPlus}) \frac{\gamma \vdash \underline{e_1} \Rightarrow \underline{n_1} \quad \gamma \vdash \underline{e_2} \Rightarrow \underline{n_2}}{\gamma \vdash \underline{e_1 + e_2} \Rightarrow \underline{n}} \text{ provided } pLUS(\underline{n_1}, \underline{n_2}, \underline{n})$$

$$(\text{CalcTimes}) \frac{\gamma \vdash \underline{e_1} \Rightarrow \underline{n_1} \quad \gamma \vdash \underline{e_2} \Rightarrow \underline{n_2}}{\gamma \vdash \underline{e_1 * e_2} \Rightarrow \underline{n}} \text{ provided } tIMES(\underline{n_1}, \underline{n_2}, \underline{n})$$

$$(\text{CalcNot}) \frac{\gamma \vdash \underline{e_1} \Rightarrow \underline{b_1}}{\gamma \vdash \neg \underline{e_1} \Rightarrow \underline{b}} \text{ provided } myNot(\underline{b_1}, \underline{b})$$

$$(\text{CalcAnd}) \frac{\gamma \vdash \underline{e_1} \Rightarrow \underline{b_1} \quad \gamma \vdash \underline{e_2} \Rightarrow \underline{b_2}}{\gamma \vdash \underline{e_1 \wedge e_2} \Rightarrow \underline{b}} \text{ provided } myAnd(\underline{b_1}, \underline{b_2}, \underline{b})$$

$$\text{CalcOr} \frac{\gamma \vdash \underline{e_1} \Rightarrow \underline{b_1} \quad \gamma \vdash \underline{e_2} \Rightarrow \underline{b_2}}{\gamma \vdash \underline{e_1 \vee e_2} \Rightarrow \underline{b}} \text{ provided } myAnd(\underline{b_1}, \underline{b_2}, \underline{b})$$

$$\begin{array}{l}
\text{(CalcEq)} \frac{\gamma \vdash \underline{e}_1 \Rightarrow \underline{a}_1 \quad \gamma \vdash \underline{e}_2 \Rightarrow \underline{a}_2}{\gamma \vdash \underline{e}_1 = \underline{e}_2 \Rightarrow \underline{b}} \text{ provided } eQ(\underline{a}_1, \underline{a}_2, \underline{b}) \\
\text{(CalcGt)} \frac{\gamma \vdash \underline{e}_1 \Rightarrow \underline{n}_1 \quad \gamma \vdash \underline{e}_2 \Rightarrow \underline{n}_2}{\gamma \vdash \underline{e}_1 > \underline{e}_2 \Rightarrow \underline{b}} \text{ provided } gT(\underline{n}_1, \underline{n}_2, \underline{b})
\end{array}$$

Note that in **(CalcVar)**, no answer is returned if $\underline{x} \notin \text{dom}(\gamma)$. The calculator gets stuck!

What about *Soundness* and *Completeness* of the *calculator* with respect to the *Definitional Interpreter*? How do those statements change?

We need a notion of a table and a valuation *agreeing with each other*. That is, for every $\underline{x} \in \text{dom}(\gamma)$: $\text{eval}[\llbracket \gamma(\underline{x}) \rrbracket] \rho = \rho(\underline{x})$

With this assumption on γ and ρ , it is fairly easy to state and prove Soundness.

Completeness is somewhat harder, and requires an additional assumption — namely that $\underline{x} \in \text{vars}(\underline{e})$, $\underline{x} \in \text{dom}(\gamma)$, where $\text{vars}(\underline{e})$ denotes the set of variables appearing in expression $\underline{E}$. This condition ensures that the calculator does not get stuck because a variable cannot be looked up in the table.

Exercise: Define the function $\text{vars}(\underline{e})$. Note the similarity in structure to `ht`, `size`, and `eval`.

Type Preservation (version 2)

The type preservation theorem requires some additional conditions (apart from the assumption that elementary operations are type-sound).

We say that a table γ is *type-consistent* with a type assumption Γ if for every $\underline{x} \in \text{dom}(\gamma)$: $\underline{x} \in \text{dom}(\Gamma)$ and $\Gamma \vdash \gamma(\underline{x}) : \Gamma(\underline{x})$. That is, the answer associated with any variable in a table is indeed of the same type associated with it by the type assumption.

Theorem (Type Preservation under $\gamma \vdash \underline{e} \Rightarrow \underline{a}$)

For all expressions $\underline{e} \in \text{Exp}$, $\underline{a} \in \text{Ans}$,
for all type assumptions Γ ,
for all tables γ type-consistent with Γ ,
for all types $\underline{\tau} \in \text{Typ}$,
if $\Gamma \vdash \underline{e} : \underline{\tau}$ and $\gamma \vdash \underline{e} \Rightarrow \underline{a}$, then $\Gamma \vdash \underline{a} : \underline{\tau}$

Proof (By Induction on the structure/*ht* of $\underline{e}$).

Base cases ($\text{ht}(\underline{e}) = 0$)

Subcases $(\underline{e} \equiv \underline{n})$ and $(\underline{e} \equiv \underline{b})$ are essentially unchanged.

There is a new base case: $\underline{e} \equiv \underline{x}$.

Assume $\Gamma \vdash \underline{e} : \underline{\tau}$. Therefore $\underline{\tau} = \Gamma(\underline{x})$. (The case of $\underline{x} \notin \text{dom}(\Gamma)$ cannot arise from the assumption).

Now $\gamma \vdash \underline{x} \implies \gamma(\underline{x})$. Since γ is assumed type-consistent with $\Gamma, \Gamma \vdash \gamma(\underline{x}) : \Gamma(\underline{x})$.
 So $\Gamma \vdash \underline{a} : \underline{\tau}$

The **Induction Hypothesis** is a suitably modified version of the earlier **IH**, and the cases in the **Induction Step** ($ht(\underline{e}) = 1 + k$) are more or less the same as before, with the appropriate changes.

Exercise: Complete this proof.

Exercise: Encode the type-checking relation $\Gamma \vdash \underline{e} : \underline{\tau}$ in PROLOG as a predicate `hastype(G, E, T)`.

Compilation and execution on a Stack Machine

The stack machine now needs to be modified to incorporate an additional component, namely the table. The configurations are now triples — a table, a stack of values and a code list.

The opcodes need only a small extension however — an opcode to look up a variable in the table. (In practice, we get rid of the variables and use some address/indexing mechanism).

```
type opcode = LDN of int | LDB of myBool | LOOKUP of string
            | PLUS | TIMES | AND | OR | NOT | EQ | GT;;
```

The `compile` function therefore has minimal changes == the inclusion on a line for compiling variables

```
let rec compile e = match e with
  Num n -> [ LDN n ]
| Bl mb -> [ LDB mb ] (* Constants *)
| V x -> [ LOOKUP x ] (* Variables *)
| Plus (e1, e2) -> (compile e1) @ (compile e2) @ [ PLUS ]
| Times (e1, e2) -> (compile e1) @ (compile e2) @ [ TIMES ]
| And (e1, e2) -> (compile e1) @ (compile e2) @ [ AND ]
| Or (e1, e2) -> (compile e1) @ (compile e2) @ [ OR ]
| Not e1 -> (compile e1) @ [ NOT ]
| Eq (e1, e2) -> (compile e1) @ (compile e2) @ [ EQ ]
| Gt(e1, e2) -> (compile e1) @ (compile e2) @ [ GT ]
;;
```

The stack machine, now endowed with a table in its configurations, needs to specify how the `LOOKUP(x)` opcode is executed. Otherwise, it is substantially the same (other than now containing a table component). Out of indolence, we have represented a table as a function from strings to *exp* (and not very precisely to answers).

```
exception Stuck of (string -> exp) * exp list * opcode list;;
```

```
let rec stkmc g s c = match s, c with
  a::[], [ ] -> a (* no more opcodes, return top *)
| s', (LDN n)::c' -> stkmc g ((Num n)::s') c'
```

```

| s', (LDB b)::c' -> stkmc g ((Bl b)::s') c'
| s', (LOOKUP x)::c' -> stkmc g ((g x)::s') c'
| (Num n2)::(Num n1)::s', PLUS::c' ->
    stkmc g (Num(n1+n2)::s') c'
| (Num n2)::(Num n1)::s', TIMES::c' ->
    stkmc g (Num(n1*n2)::s') c'
| (Bl b2)::(Bl b1)::s', AND::c' ->
    stkmc g (Bl(myAnd b1 b2)::s') c'
| (Bl b2)::(Bl b1)::s', OR::c' ->
    stkmc g (Bl(myOr b1 b2)::s') c'
| (Bl b1)::s', NOT::c' -> stkmc g (Bl(myNot b1)::s') c'
| (Num n2)::(Num n1)::s', EQ::c' ->
    stkmc g (Bl (bool2myBool(n1 = n2))::s') c'
| (Num n2)::(Num n1)::s', GT::c' ->
    stkmc g (Bl (bool2myBool(n1 > n2))::s') c'
| _, _ -> raise (Stuck (g, s, c))
;;

```

The exception `Stuck` now takes 3 arguments, namely the table, stack and opcode list. The only new line is

```

| s, (LOOKUP x)::c' -> stkmc g ((g x)::s) c'

```

where the value obtained from the table, namely `(g x)`, is pushed onto the stack.